UbuntuTheSkillTrainer: Web Technologies Team Project

Aisha N Chihuri

Rachel Murambiwa

Soukouratou Karim Souleymane

Tanatswa Mhiribidi

Ashesi University

CS341: Web Technologies

Mr. Kwadwo Osafo-Maafo

November 30, 2025

UbuntuTheSkillTrainer: Web Technologies Team Project

Introduction

Ubuntu is a platform that offers interactive lessons and tracking tools to help users develop skills in various fields, such as crocheting, henna design, traditional music, particularly the mbira, and bead accessory making, ensuring steady progress and confidence-building. Ubuntu, the skill trainer platform, provides modules, videos, and quizzes designed to help build accessible, engaging, and effective learning experiences for learners of all ages.

Business Description

Ubuntu Skills functions as an educational service that provides Interactive online courses, Hands-on lessons and resources, Progress dashboards, and certificates. Its purpose is to empower individuals to gain real-world, creative, and technical skills through a supportive and user-friendly learning environment.

Problem with the Website Solved

Many people want to learn something new, to build skills that bring joy or opportunity, yet they often feel lost. Finding simple, beginner-friendly lessons is difficult, and motivation fades. Many lack practical guidance that truly meets them at their level and wish they could learn at their own pace without feeling rushed or confused. Ubuntu steps in to ease these struggles by offering a friendly and structured learning platform filled with engaging videos, helpful images, and hands-on activities that make learning feel achievable. Through interactive quizzes and a personalised progress dashboard, learners can monitor their growth and celebrate every small victory. Ubuntu turns learning from a lonely, confusing journey into one that is guided, encouraging, and confidence-building, bringing clarity where there was overwhelm and hope where there was frustration.

Types of Users

The platform will be used by several types of users:

- Learners (Students)
  Sign up, log in, and enroll in courses
  Access lessons, quizzes, and resources for each course enrolled in
  Track progress on their dashboard
  Earning certificates

- Instructors
  Create new courses or modules
  Upload videos, images, and learning materials
  Add quizzes and update course content
- Administrators
  Manage users (approve, deactivate, support)
  Oversee content and system settings
  Monitor platform performance
- Ensure data security and updates

Important links

CMS: - https://ubuntuthetrainer.wordpress.com/?_gl=1*1f02y70*_gcl_au*MTQzOTY5NDg5Ny4xNzYwMTMwNTgw

GITHUB:

https://github.com/AishaNoku/UbuntuTheSkillTrainer.git

LINK TO LIVE SERVER :

http://169.239.251.102:341/~rachel.murambiwa/UbuntuTheSkillTrainer/index.php

Functionality

The major functions that were built for the problems articulated are:

- HashPassword: Ensures security by hashing user passwords before storing them in the database, so the exact password is never visible.
- gradeModuleQuiz(): Calculates and records a learner's score for each module quiz.
- GenerateCertificate: Automatically generates a certificate for the user upon completing the course.
- function getSecureDBConnection() - Creates PDO connection with security settings
- function initSecureSession() - Configures secure session settings (httponly, secure, samesite).
- function validateSession() - Checks user agent matches stored value

Architecture and design

The system follows a 3-tier architecture, separating concerns into Presentation, Logic, and Data layers:

1. Presentation Layer (Frontend):
   - Built with HTML, CSS, and JavaScript.
   - Responsible for user interaction, displaying course modules, quizzes, progress bars, and certificates.
   - Handles events like module switching, quiz submission, and marking modules as complete.
2. Logic / Application Layer (Backend):
   - Validates user input and ensures the application behaves as expected using Php.
   - Can be implemented in a server-side language
3. Data Layer (Database):
   - Stores user data, hashed passwords, courses, user enrolment
   - Ensures data is persistently stored and retrieved securely.

Modules / Layers of Implementation

The system is organised into functional modules that correspond to the layers:

1. User Management Module:
   Handles user registration, login, and password hashing.
   Stores user progress and completion data.
2. Course Module:
   Displays course content and multimedia resources.
   Allow learners to navigate between modules.
3. Quiz Module:
   Implements quiz questions and grading functions.
4. Certificate Module:
   Generates certificates automatically when users complete all modules.
5. Progress Tracking Module:
   Tracks completion of each module and updates the progress bar dynamically.

Individual Contributions

Tanatswa C Mhiribidi

1. Role: Backend lead and full-stack developer responsible for the beadmaking course.
2. Key Contribution and Role: My primary responsibility was transforming our group's static HTML prototype into a fully functional, dynamic web application. As the backend lead, I initiated the server-side infrastructure using PHP and MySQL. I was responsible for setting up the local development environment using XAMPP, creating the relational database schema, and implementing a secure user authentication system. Additionally, I developed a Bead Making Course, implementing interactive features such as progress tracking, end of module quizzes and an end-of-course generatable certificate of completion.
3. Features Worked On and Completed: Secure user authentication system: I developed the full registration and login lifecycle, including server-side validation for strong passwords using Regex. I also implemented session management using PHP to persist user state across pages. Finally, under this, I created a dynamic header that automatically toggles between "login" and "logout" based on the user's state. Gating Logic: I implemented access control logic that restricts unregistered users from viewing course content.
4. Most Important Algorithm or Design: My most significant technical contribution was the development of a resilient database connection algorithm to resolve conflicts in team infrastructure. It solved the problem of port conflicts, which were causing constant crashes when sharing code via git. The algorithm was a connection script that first attempted to use the standard default port, and then, if an error occurred with that, it would attempt a connection to the custom port. The script only terminated if both ports failed.
5. Challenges

- Port Conflicts and Database Synchronization: A major challenge was that my local XAMPP environment ran MySQL on a custom port (3307) while my teammates used the default port (3306). This caused database connection failures when sharing code. I resolved this issue by writing a db_connect.php script that automatically attempts multiple connection ports, allowing the code to run seamlessly on any machine without manual editing.

- Git Merge Conflicts: We faced “diverged” branch issues when attempting to merge database configuration files. I learnt to resolve these by aborting stuck merges using git merge - abort and stashing local changes.

6. Lessons Learnt

- The importance of prepared statements: I learnt that standard SQL queries leave a system vulnerable to injection attacks thus the importance of separating SQL logic from user data to ensure security.
- State Management: I learnt how to use sessions to carry a user’s identity from page to page, which is fundamental to interactive web applications.

Soukouratou Karim

1. Role: Developer for both frontend and backend features for the Henna Course. Responsible for writing the project description document.
2. Feature Work:
   Modules Implementation: I successfully developed three engaging modules for the Henna Course. At the conclusion of each module, users are prompted to take a quiz that assesses their understanding of the material. Additionally, users can track their progress throughout the course, providing them with a sense of achievement and motivation. Navigation Enhancement: I created a user-friendly button that allows participants to seamlessly transition from one module to the next upon completion, enhancing the learning experience with smooth navigation.
3. I wrote a comprehensive project description outlining the key objectives, the challenges our team aimed to address, and the innovative solutions we are implementing to enhance the learning experience for users. I designed a quizzes section that includes multiple-choice questions (MCQs) and true/false questions. This component serves as a crucial tool to evaluate users’ grasp of the concepts taught in the modules.
4. Algorithm:

- Module Quiz Grade Calculation: I developed an algorithm specifically for calculating the grade for each module's quiz, ensuring accurate assessment of the users' performance.

5. Challenges:

- During the implementation process, I encountered several challenges, particularly on the frontend. One significant hurdle was achieving a design that closely matched my

vision for the course interface. Additionally, I faced difficulties with the progress bar functionality; I aimed to create a feature that would display a precise percentage of progress as users completed each module, but was unable to accomplish this aspect effectively.

Rachel Murambiwa

1. Role: Developer responsible for both frontend and backend features of the Crochet Learning Platform, including creating the UML diagram.
2. Implemented module progress tracking with per-module completion and an overall progress bar for the Crochet skill using localStorage. Created module navigation with clickable cards and previous/next buttons, including hover effects. Connected homepage buttons to their respective module pages. Added email validation for registration with inline messages and PHP server-side checks. Styled UI elements (buttons, progress bars, module cards) to achieve a consistent and interactive interface.
3. Updated responsiveness across multiple pages for better cross-device usability. Created the UML diagram for the database structure using PlantUML. Deployed the full website to the hosting server using CyberDuck.
4. Testing Strategies:
   Performed manual testing of module progress, navigation, and form validation. Verified that progress persists correctly across reloads and user sessions.
5. Key Algorithm/Design:
   Weighted progress calculation that dynamically sums the weights of completed modules to compute overall course completion.
6. Challenges:
   In managing page paths within a GitHub-style folder structure, I faced challenges with linking, prompting a deeper exploration of relative and absolute paths. This understanding helped me resolve linking issues. I also implemented a weighted module progress system to track user engagement, ensuring real-time updates to the progress meter after each module's completion. To enhance usability and prevent errors, I disabled the "Mark as Complete" buttons after they were clicked, maintaining the integrity of progress data. Additionally, I coordinated client-side and server-side validation to ensure security while providing a smooth user experience, creating an intuitive system for users to navigate confidently.

7. Lessons Learned:
   Using JSON structures in localStorage improves scalability and maintainability of stored data. Combining client-side and server-side validation ensures both robust security and better UX. Breaking features into modular, testable components simplifies debugging and enhances development clarity. Gained experience deploying a website using CyberDuck, including managing directories and server-side file updates.

Aisha N Chihuri

1. Role and Responsibilities
   Database Security Developer and Mbira Course Integration Specialist at Ubuntu Skills, implemented protective security measures for the platform, and developed an interactive Mbira tutorial course using JavaScript. Scribe for the final documentation and socumentation checks.
2. Database Design and Security Features:
   I designed the database structure with tables to organize the platform's data. The users table holds account information, including security features like account_status to lock suspicious accounts and failed_login_attempts to track incorrect password attempts. For password security, I used Argon2id hashing, which converts passwords into unreadable codes. I configured it to use 64MB of memory and run four iterations, making it hard for attackers to crack passwords even if they access the database. To protect against SQL injection, I used prepared statements for every database query, ensuring user input is treated as data only. I also implemented CSRF protection by generating unique, random tokens for each session, validating them with form submissions, and expiring them after one hour.For session management, I store the user's browser user agent string at login and check it on every page visited. If the user agent changes, I log the user out. Sessions automatically expire after thirty minutes of inactivity.
3. Mbira Course Development and JavaScript Implementation:
   I researched traditional Zimbabwean mbira music and compiled educational videos from YouTube. I organized this into six modules, starting with beginner techniques and progressing to advanced songs. Module 0 introduces the mbira and its cultural significance, Module 1 covers basic techniques, and later modules explore songs like Karigamombe and Chemutengure. I developed an interactive course system using

JavaScript, creating a data object called courseData that contains all six modules, each with lessons that include titles, descriptions, embedded videos, and quiz questions. Functions are in place to display content dynamically based on lesson selection, ensuring a smooth single-page experience. For progress tracking, I implemented a system that maintains arrays for completed lessons and stores quiz scores. Clicking "Mark as Complete" adds the lesson ID to the completedLessons array, updates a progress chart, and saves data to the PHP backend. I used Chart.js to visualize course completion with a doughnut chart. Additionally, I created an interactive quiz at the end of each module. JavaScript assesses student answers, calculates scores, and gives immediate feedback. Quiz results are stored in both the JavaScript state object and the database.

4. Challenges and Solutions

   I faced challenges with session hijacking detection. To avoid logging out legitimate users, I stored the browser's user agent at first login and compared it on subsequent requests, destroying the session only on a clear mismatch. Maintaining performance while adding security features was another hurdle. I optimized password hashing and action logging by adding database indexes to speed up queries and implementing automated cleanup events to manage table sizes. With the JavaScript modules, I struggled to keep track of student progress after page refreshes since data was stored in temporary variables. I resolved this by using localStorage to save the completedLessons array and also sending this data to the PHP backend for permanent storage. My XAMPP setup used MySQL on port 3307, while my teammates were on 3306, leading to connection issues and we also faced merge conflicts with diverged branches in database config files. I learned to handle this by aborting troublesome merges with `git merge --abort`for local changes.

5. What I Learned

   During this project, I faced challenges embedding YouTube videos in course pages. Initially, I only had URLs stored in the courseData object, but I needed a way for students to watch videos on the platform. I resolved this by adding a "frame" property to each lesson, which contains the iframe HTML code. I also improved my JavaScript code organization by separating course data, display logic, and state management into distinct functions, making debugging easier. I learned that security must be integrated from the start to be effective. Using prepared statements eliminated SQL injection risks, and good security logging was essential for debugging and monitoring attacks.

My experience with session security reinforced the need for multiple protection layers to secure user accounts effectively. Overall, this project demonstrated how effective database design, security measures, and frontend functionality collaborate to create a secure and user-friendly learning platform.

Ethical Use of AI in This Project

Throughout the development of the Ubuntu Skills platform, our team used artificial intelligence tools as learning assistants and code helpers while maintaining ethical practices. We used AI to help us understand complex concepts like security implementations, database relationships, and frontend design patterns. AI assisted us in learning about password hashing algorithms, prepared statements for SQL injection prevention, session management best practices, and modern JavaScript techniques for interactive course modules.

When implementing the course modules for Mbira, Henna, Crochet, and Bead Making, we consulted AI for syntax help, debugging strategies, and technical solutions to specific problems like progress tracking and quiz scoring algorithms. However, we personally researched the cultural and educational aspects of each craft and made all content decisions ourselves.

We believe AI should be used as an educational tool that enhances learning rather than replaces critical thinking and creativity. Throughout this project, we maintained academic integrity by using AI as a teacher and reference guide, not as a replacement for our own understanding and effort. Every line of code we submitted represents our actual comprehension of web development, security implementation, and database design principles, validated through extensive testing and our ability to explain and defend our technical decisions.

References

Acunetix. (2024). *Cross-site request forgery (CSRF) attack and prevention: Understanding and mitigating CSRF vulnerabilities in web applications*. Acunetix Web Security Zone. https://www.acunetix.com/websitesecurity/csrf-attacks/

Argon2 Development Team. (2015). *Argon2: The memory-hard function for password hashing and other applications*. Password Hashing Competition Winner. GitHub Repository. https://github.com/P-H-C/phc-winner-argon2

Chart.js Contributors. (2024). *Chart.js: Simple yet flexible JavaScript charting library for designers and developers* (Version 4.4.0). Official Documentation. https://www.chartjs.org/docs/latest/

Coursera. (2024). *Best practices for online course design: Evidence-based strategies for engaging digital learning experiences*. Coursera Career Resources. https://www.coursera.org/articles/online-course-design

freeCodeCamp.org. (2024). *Full stack web development curriculum: Free coding bootcamp covering HTML, CSS, JavaScript, React, Node.js, Python, and databases*. https://www.freecodecamp.org/learn

OWASP Foundation. (2024). *OWASP Zed Attack Proxy (ZAP): Comprehensive web application security scanner documentation* (Version 2.14.0). https://www.zaproxy.org/docs/

Stack Overflow Community. (2024). *PHP session security best practices: Community-curated questions and answers on secure session handling*. Stack Exchange Network. https://stackoverflow.com/questions/tagged/php+sessions